Name:- Mansi Narkhede

PRN NO :- 202501050027

Branch :- Chemical Engineering

Batch:- CH2

Subject :- EDS

Problem statements:-

4.1.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Code :-

```
import pandas as pd

# Take inputs from the user to create a list of numbers

numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers

series = pd.Series(numbers)

# Grouping by even and odd numbers and calculating the mean

grouped =series.groupby(series % 2 == 0).mean()

# Display the mean of even and odd numbers with labels

grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]

print("Mean of even and odd numbers:")

print(grouped)
```

Explorer

Average time 0.005 s 5.50 ms

Maximum time 0.015 s 15.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Debugger

Test case 1 15 ms Debug

Expected output	Actual output
1 2 3 4 5 6 7 8 9 10	1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:	Mean of even and odd numbers:
Odd ····5.0	Odd ····5.0
Even ····6.0	Even ····6.0
dtype: float64	dtype: float64

Test case 2 4 ms Debug

Expected output	Actual output
4 4 4 6 6 2 2 2 10 12 14 16	4 4 4 6 6 2 2 2 10 12 14 16
Mean of even and odd numbers:	Mean of even and odd numbers:
Even ····6.833333	Even ····6.833333
dtype: float64	dtype: float64

Test case 3 3 ms Debug

Expected output	Actual output
1 5 7 9 11 13 15 17 111 21 25	1 5 7 9 11 13 15 17 111 21 25
Mean of even and odd numbers:	Mean of even and odd numbers:
Odd ····21.363636	Odd ····21.363636
dtype: float64	dtype: float64

Problem statement :-

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

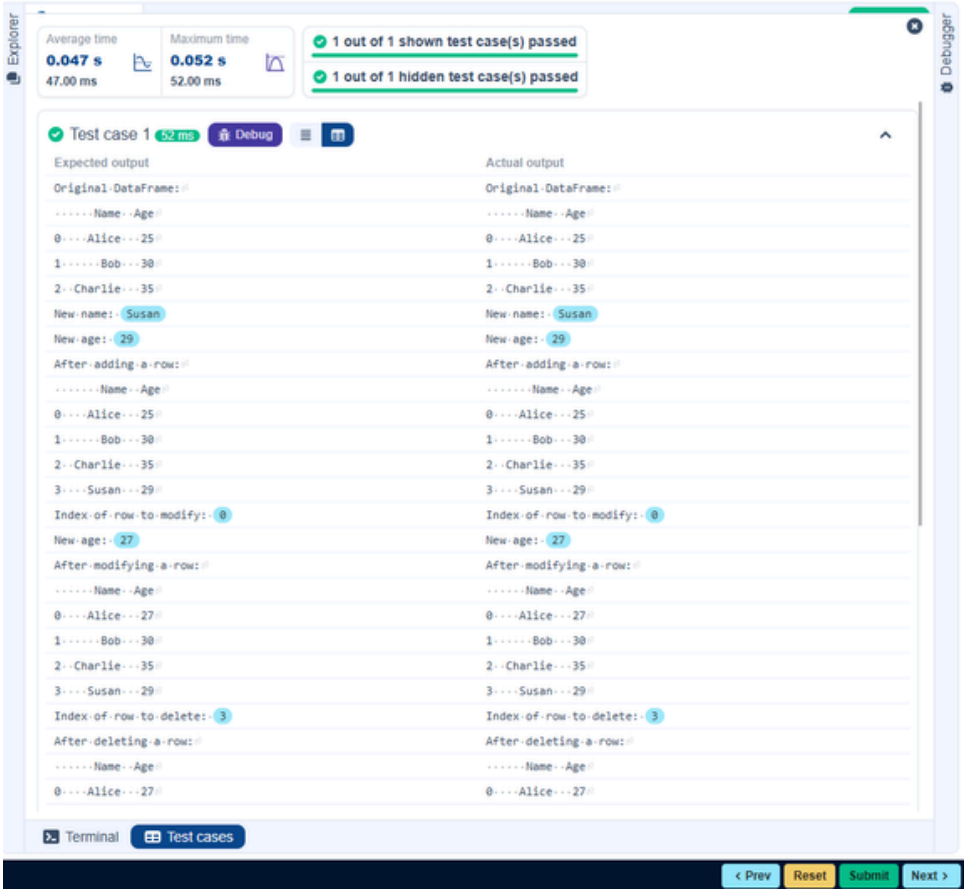
Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

Code:

```
import pandas as pd


# Provided dictionary of lists

data = {

    'Name': ['Alice', 'Bob', 'Charlie'],

    'Age': [25, 30, 35],

}


# Convert the dictionary to a DataFrame

df = pd.DataFrame(data)


# Display the original DataFrame

print("Original DataFrame:")

print(df)


# Adding a new row

name = input("New name: ")

age = int(input("New age: "))

new_row = {"Name": name,"Age": age}

df = df.append(new_row, ignore_index = True)


# Display the DataFrame after adding a new row

print("After adding a row:\n",df)


# Modifying a row

ind = int(input("Index of row to modify: "))

age = int(input("New age: "))


df.loc[ind, "Age"] = age



# Display the DataFrame after modifying a row

print("After modifying a row:")

print(df)


# Deleting a row

ind = int(input("Index of row to delete: "))


df = df.drop(index = ind).reset_index(drop = True)


# Display the DataFrame after deleting a row

print("After deleting a row:")
```

```
print(df)

# Adding a new column
gender = input("Enter genders separated by space: ").split()
df["Gender"] = gender


# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)

# Modifying a column
column = []

for i in df["Name"]:
column.append(i.upper())

df["Name"] = column
# Display the DataFrame after modifying a column
print("After modifying a column:")
print(df)

# Deleting a column
df = df.drop(columns = ["Age"])
# Display the DataFrame after deleting a column
print("After deleting a column:")
print(df)
```



Problem statement :-

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is **'B'**).

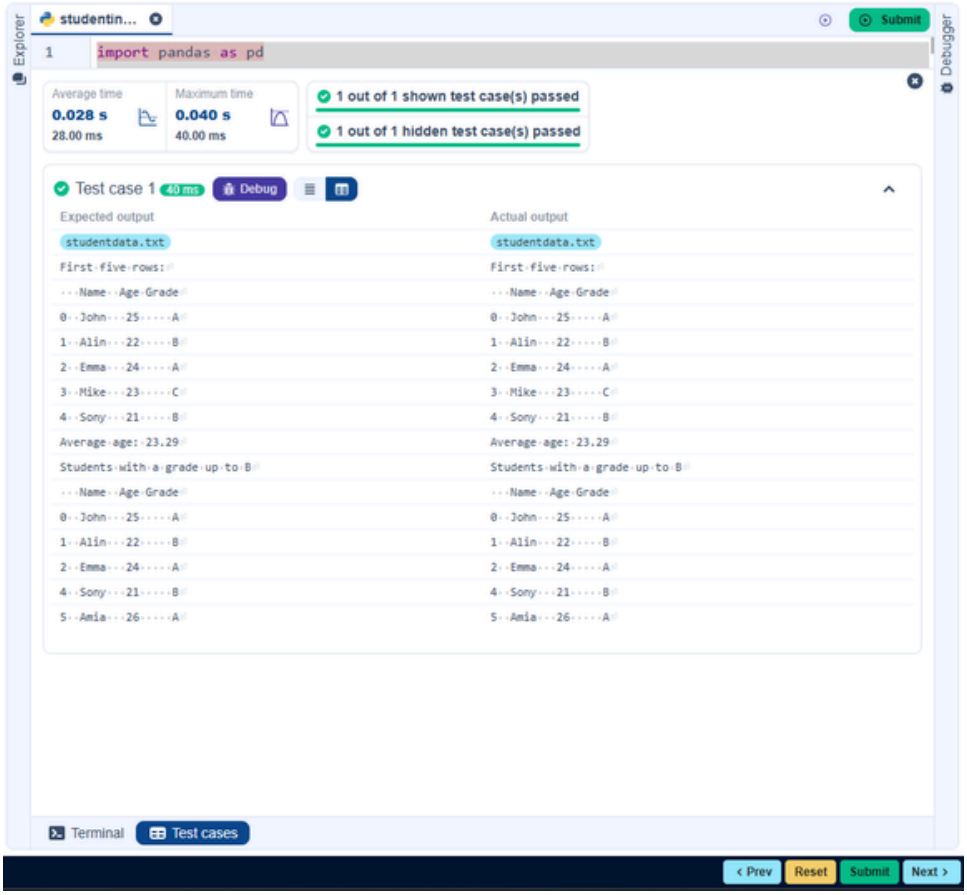
Note:

Refer to the displayed test cases for better understanding.

-import pandas as pd

```
# Read the text file into a DataFrame

file = input()

data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])



# write your code here..

print("First five rows:")

print(data.head())


average_age=round(data["Age"].mean(),2)

print(f"Average age: {average_age}")


filtered_student = data[data["Grade"] <= "B"]

print("Students with a grade up to B")
```

print(filtered_student)



Problem statement:-

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product **A**,5,20,New York

2025-01-01,Product **B**,3,15,Los Angeles

2025-01-02,Product **A**,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product **B**,2,15,Chicago

2025-01-03,Product **A**,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product **B**,5,15,Los Angeles

2025-01-05,Product **A**,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

import pandas as pd

Prompt the user for the file name

file_name = input()

```
# Load the data

df = pd.read_csv(file_name)



# Find the month with the highest total sales

df['Date'] = pd.to_datetime(df["Date"])

df["Month"] = df["Date"].dt.strftime("%Y-%m")

df["Total Sales"]=df["Quantity"] * df["Price"]


# Find the month with the highest total sales

sales_by_month=df.groupby("Month")["Total Sales"].sum()

best_month = sales_by_month.idxmax()

highest_sales = sales_by_month.max()



print(f"Best month: {best_month}")

print(f"Total sales: ${highest_sales:.2f}")
```

Problem statement:-

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product **A**,5,20,New York

2025-01-01,Product **B**,3,15,Los Angeles

2025-01-02,Product **A**,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product **B**,2,15,Chicago

2025-01-03,Product **A**,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-**01**-04,Product **B**,5,15,Los Angeles

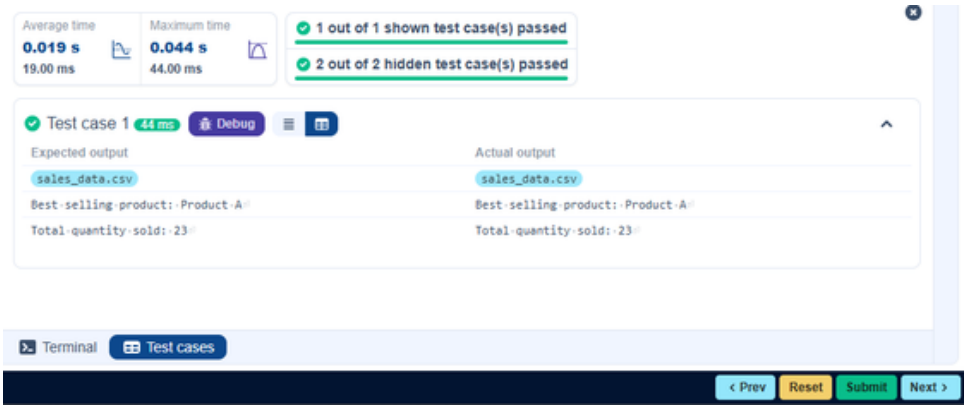
2025-01-05,Product **A**,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

```
import pandas as pd
```

code :-

```
# Prompt the user for the file name

file_name = input()


# Load the data

df = pd.read_csv(file_name)

product_sales = df.groupby("Product")["Quantity"].sum()

# Find the product with the highest total quantity sold

best_product =product_sales.idxmax()

highest_quantity = product_sales.max()


# Find the product with the highest total quantity sold


# Display the result

print(f"Best selling product: {best_product}")

print(f"Total quantity sold: {highest_quantity}")
```



Problem statements :-

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product **A**,5,20,New York

2025-01-01,Product **B**,3,15,Los Angeles

2025-01-02,Product **A**,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product **B**,2,15,Chicago

2025-01-03,Product **A**,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

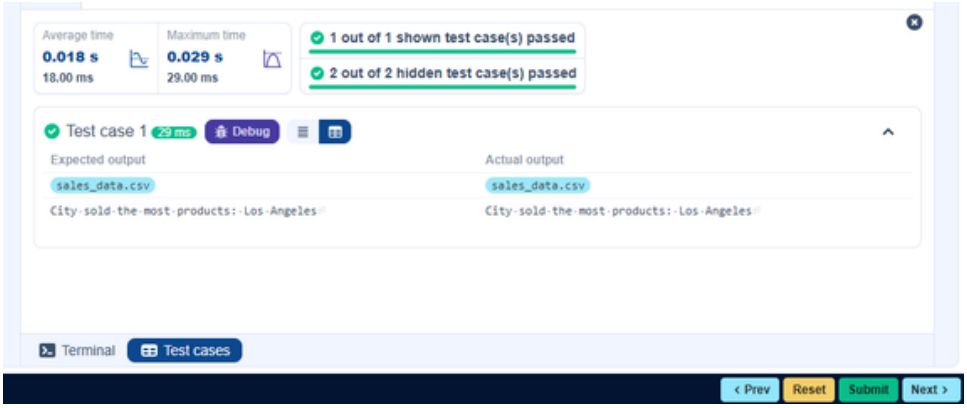
2025-01-04,Product **B**,5,15,Los Angeles

2025-01-05,Product **A**,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Code :-

```
import pandas as pd


# Prompt the user for the file name

file_name = input()


# Load the data

df = pd.read_csv(file_name)


# write the code..

city_sales = df.groupby("City")["Quantity"].sum()

best_city=city_sales.idxmax()

# Display the result

print(f"City sold the most products: {best_city}")
```



Problem statement :-

4.2.4. Most Frequently Sold Product Pairs

00:34

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product **A**,5,20,New York

2025-01-01,Product **B**,3,15,Los Angeles

2025-01-02,Product **A**,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product **B**,2,15,Chicago

2025-01-03,Product **A**,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product **B**,5,15,Los Angeles

2025-01-05,Product **A**,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Explanation:

Transactions:

- **2025-01-01:** Product A, Product B

- **2025-01-02:** Product A, Product C
- **2025-01-03:** Product B, Product A
- **2025-01-04:** Product C, Product B
- **2025-01-05:** Product A, Product C

Now, let's count how often the pairs of products appear together:

- **Product A and Product B**: Appear in transactions on 2025-01-01 and 2025-01-03.
- **Product A and Product C**: Appear in transactions on 2025-01-02 and 2025-01-05.
- **Product B and Product C**: Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

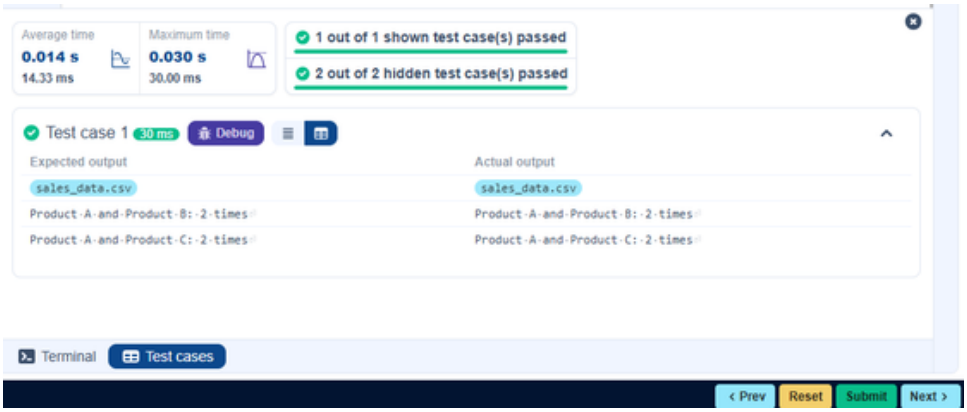
- **Product A and Product B** (2 times)
- **Product A and Product C** (2 times)

Code :-

```
import pandas as pd

from itertools import combinations

from collections import Counter



# Prompt user to input the file name

file_name = input()



# Read data from the specified CSV file

df = pd.read_csv(file_name)

# write the code



# write the code

date_products = {}



for date, group in df.groupby('Date'):

    products = group['Product'].unique()

    if len(products) > 1:

        date_products[date] = products



# Count product pairs

pair_counter = Counter()



for products in date_products.values():

    # Sort to avoid duplicate pairs like (A, B) and (B, A)

    pairs = combinations(sorted(products), 2)

    pair_counter.update(pairs)



# Find the maximum frequency

if pair_counter:

    max_count = max(pair_counter.values())



    # Output the most frequent product pairs

    for pair, count in pair_counter.items():

        if count == max_count:

            print(f"{pair[0]} and {pair[1]}: {count} times")

else:

    print("No product pairs found.")
```

Output the most frequent product pairs

Output the most frequent product pairs



Problem ststements :-

4.2.5. Titanic Dataset Analysis and Data Cleaning

00:21

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using .info()).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

- titanicDataAnalysis.py

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
# 1. Display the first 5 rows of the dataset
print(data.head())
```

```
# 2. Display the last 5 rows of the dataset
print(data.tail())

# 3. Get the shape of the dataset (number of rows and columns)
print(data.shape)

# 4. Get a summary of the dataset
print(data.info())

# 5. Get basic statistics of the dataset
print(data.describe())

# 6. Check for missing values and display the count
print(data.isnull().sum())
# 7. Fill missing values in the ‘Age’ column with the median age

data['Age'].fillna(data['Age'].median(), inplace=True)

# 8. Fill missing values in the ‘Embarked’ column with the most frequent value (mode)

data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

# 9. Drop the ‘Cabin’ column due to many missing values
data.drop(columns=['Cabin'], inplace=True)

# 10. Create a new column 'FamilySize' by adding ‘SibSp’ and ‘Parch’
data['FamilySize'] = data['SibSp'] + data['Parch']
```

Translation Failed

Reason for late submission

Please enter at least 15 characters

```
import pandas as pd
import numpy as np
```

code :-

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
# 1. Display the first 5 rows of the dataset
print(data.head())

# 2. Display the last 5 rows of the dataset
print(data.tail())

# 3. Get the shape of the dataset (number of rows and columns)
print(data.shape)
```

4. Get a summary of the dataset

print(data.info())

5. Get basic statistics of the dataset

print(data.describe())

6. Check for missing values and display the count

print(data.isnull().sum())

7. Fill missing values in the ‘Age’ column with the median age

data['Age'].fillna(data['Age'].median(), inplace=True)

8. Fill missing values in the ‘Embarked’ column with the most frequent value (mode)

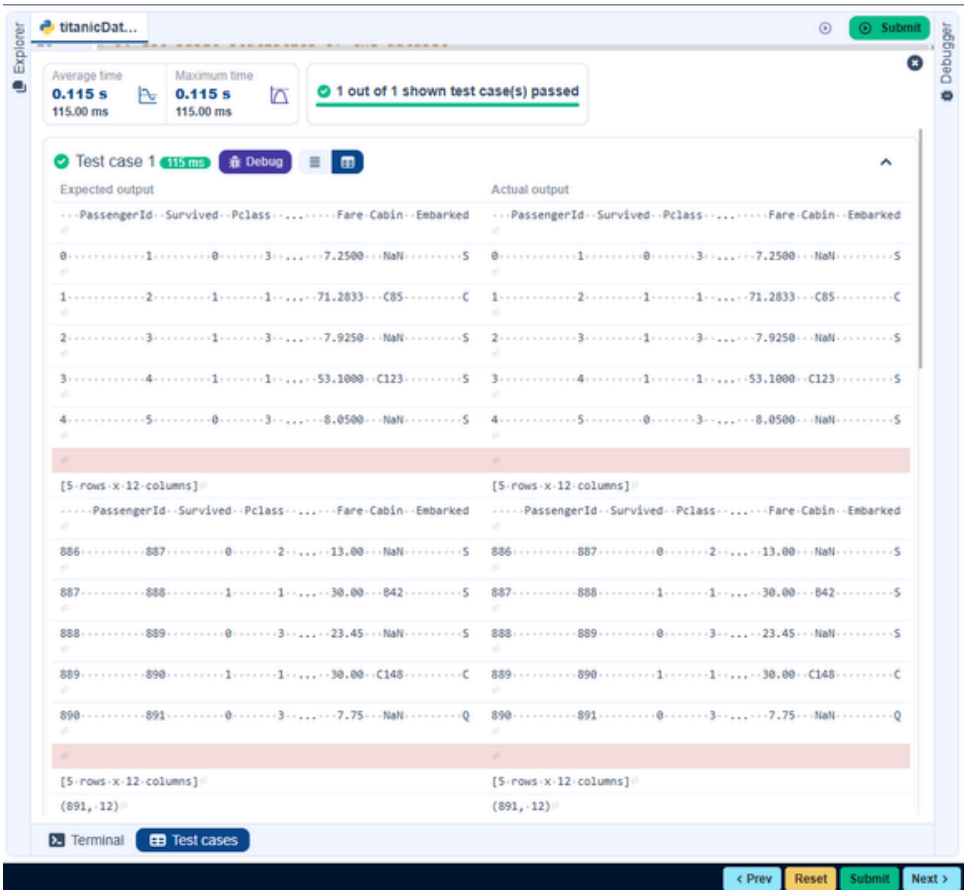
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

9. Drop the ‘Cabin’ column due to many missing values

data.drop(columns=['Cabin'], inplace=True)

10. Create a new column 'FamilySize' by adding ‘SibSp’ and ‘Parch’

data['FamilySize'] = data['SibSp'] + data['Parch']



Problem statements :-

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column ‘IsAlone’ which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the ‘Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the ‘Embarked’ column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

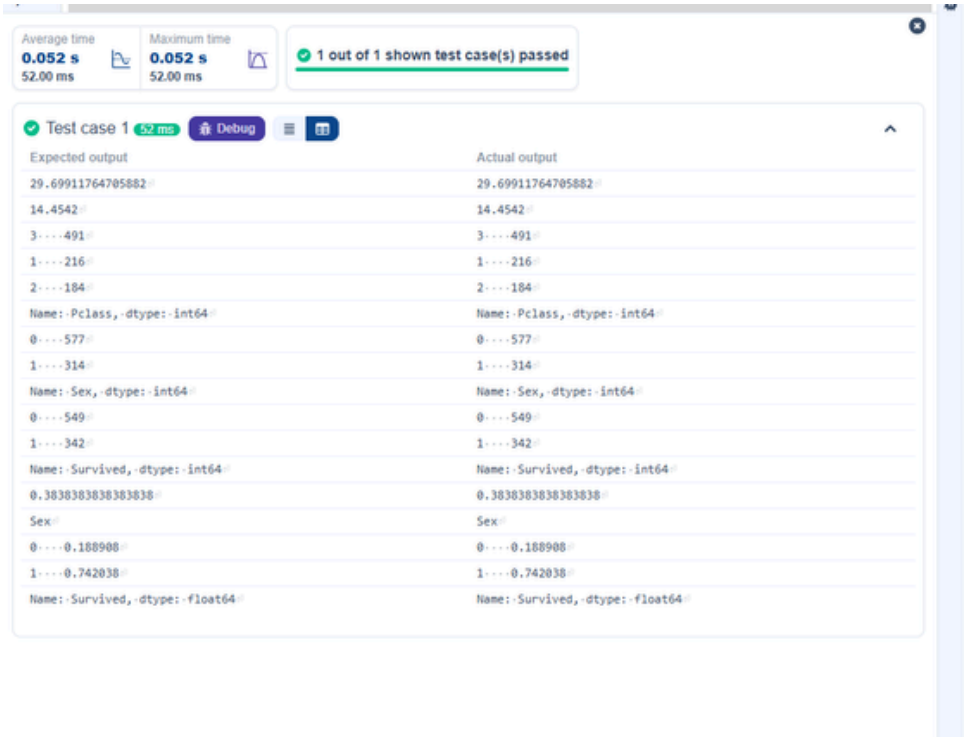
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

```
import pandas as pd

import numpy as np


# Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

data['FamilySize'] = data['SibSp'] + data['Parch']


# 1. Create a new column ‘IsAlone' (1 if alone, 0 otherwise)

data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)


# 2. Convert ‘Sex' to numeric (male: 0, female: 1)

data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})


# 3. One-hot encode the ‘Embarked' column

data = pd.get_dummies(data, columns=['Embarked'])


# 4. Get the mean age of passengers

mean_age = data['Age'].mean()

print( mean_age)

# 5. Get the median fare of passengers

median_fare = data['Fare'].median()

print( median_fare)


# 6. Get the number of passengers by class
```

```
print( data['Pclass'].value_counts())


# 7. Get the number of passengers by gender

print( data['Sex'].value_counts())


# 8. Get the number of passengers by survival status

print( data['Survived'].value_counts())

# 9. Calculate the overall survival rate

survival_rate = data['Survived'].mean()

print(format(survival_rate))


# 10. Calculate the survival rate by gender

survival_by_gender = data.groupby('Sex')['Survived'].mean()

print( survival_by_gender)
```



Problem ststement:-

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Code :-

```
import pandas as pd

import numpy as np


# Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

data['FamilySize'] = data['SibSp'] + data['Parch']

data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Calculate the survival rate by class



# 2. Calculate the survival rate by embarked location



# 3. Calculate the survival rate by family size



# 4. Calculate the survival rate by being alone



# 5. Get the average fare by class



# 6. Get the average age by class



# 7. Get the average age by survival status



# 8. Get the average fare by survival status
```

```
# 9. Get the number of survivors by class



# 10. Get the number of non-survivors by class
```


```
# 10. Get the number of non-survivors by class
```


```
print(data.groupby('Pclass') ['Survived'].mean())


#2. Calculate the survival rate by embarked location (Embarked_S)


print(data.groupby ('Embarked_S') ['Survived'].mean())


#3. Calculate the survival rate by family size


print(data.groupby('FamilySize') ['Survived'].mean())


#4. Calculate the survival rate by being alone


print(data.groupby('IsAlone') ['Survived'].mean())


#5. Get the average fare by class
print(data.groupby('Pclass') ['Fare'].mean())


#6. Get the average age by class


print(data.groupby('Pclass') ['Age'].mean())


#7. Get the average age by survival status


print(data.groupby('Survived')['Age'].mean())


#8. Get the average fare by survival status


print(data.groupby('Survived') ['Fare'].mean())


#9. Get the number of survivors by class (sort by values descending)
print(data[data['Survived'] == 1] ['Pclass'].value_counts())


#10. Get the number of non-survivors by class (sort by values descending)


print(data[data['Survived'] == 0] ['Pclass'].value_counts())
```

Explorer

Average time
0.063 s
63.00 ms

Maximum time
0.063 s
63.00 ms

1 out of 1 shown test case(s) passed

Debugger

Test case 1 63 ms Debug

Expected output	Actual output
Pclass	Pclass
1 0.629630	1 0.629630
2 0.472826	2 0.472826
3 0.242363	3 0.242363
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Embarked_S	Embarked_S
0 0.506073	0 0.506073
1 0.336957	1 0.336957
Name: Survived, dtype: float64	Name: Survived, dtype: float64
FamilySize	FamilySize
0 0.303538	0 0.303538
1 0.552795	1 0.552795
2 0.578431	2 0.578431
3 0.724138	3 0.724138
4 0.200000	4 0.200000
5 0.136364	5 0.136364
6 0.333333	6 0.333333
7 0.000000	7 0.000000
10 0.000000	10 0.000000
Name: Survived, dtype: float64	Name: Survived, dtype: float64
IsAlone	IsAlone
0 0.505650	0 0.505650
1 0.303538	1 0.303538
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Pclass	Pclass
1 84.154687	1 84.154687

Terminal Test cases

‹ Prev Reset Submit Next ›

Problem ststement:-

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

00:21

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

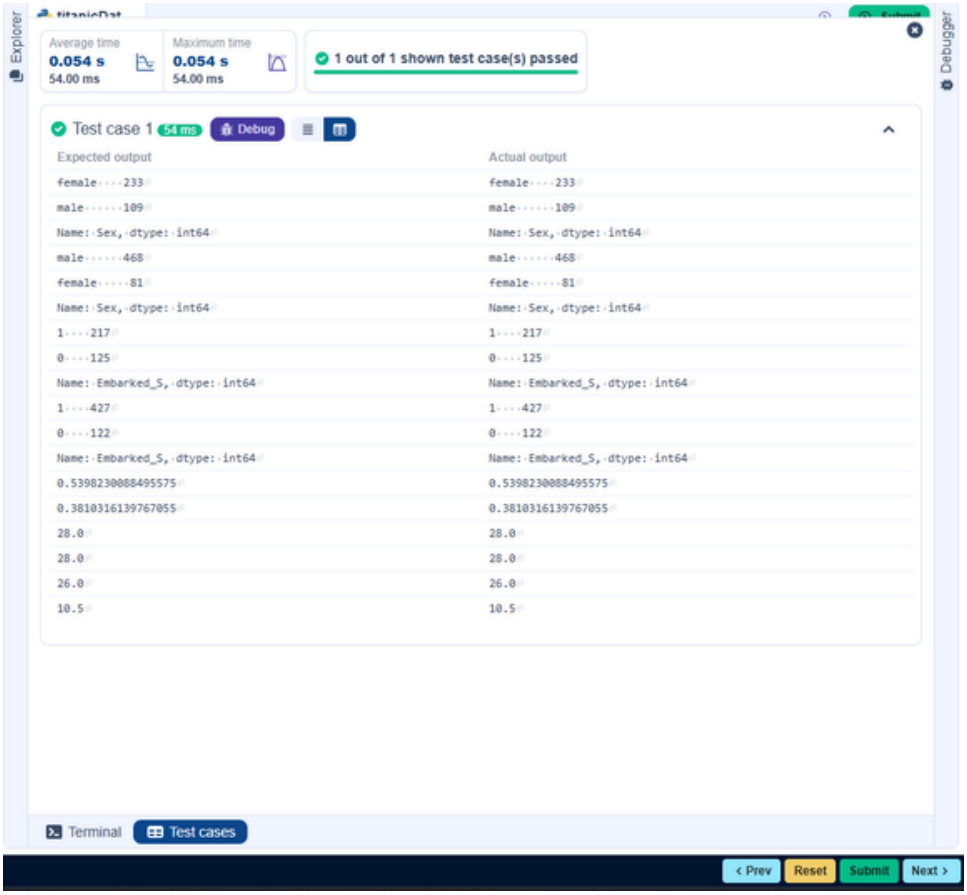
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Code :-

```
import pandas as pd

import numpy as np


# Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)


# 1. Get the number of survivors by gender

survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()

print(survivors_by_gender)


# 2. Get the number of non-survivors by gender

non_survivors_by_gender = data[data['Survived'] == 0]['Sex'].value_counts()

print(non_survivors_by_gender)

# 3. Get the number of survivors by embarked location (Embarked_S)

survivors_by_embarked_s = data[data['Survived'] == 1]['Embarked_S'].value_counts()

print(survivors_by_embarked_s)


# 4. Get the number of non-survivors by embarked location (Embarked_S)

non_survivors_by_embarked_s = data[data['Survived'] == 0]['Embarked_S'].value_counts()

print(non_survivors_by_embarked_s)


# 5. Percentage of children (Age < 18) who survived

children = data[data['Age'] < 18]

children_survival_rate = children['Survived'].mean()

print(children_survival_rate)

# 6. Percentage of adults (Age >= 18) who survived

adults = data[data['Age'] >= 18]

adults_survival_rate = adults['Survived'].mean()

print(adults_survival_rate)


# 7. Median age of survivors

median_age_survivors = data[data['Survived'] == 1]['Age'].median()

print(median_age_survivors)


# 8. Median age of non-survivors

median_age_non_survivors = data[data['Survived'] == 0]['Age'].median()

print(median_age_non_survivors)

# 9. Median fare of survivors

median_fare_survivors = data[data['Survived'] == 1]['Fare'].median()

print(median_fare_survivors)
```

10. Median fare of non-survivors

```
median_fare_non_survivors = data[data['Survived'] == 0]['Fare'].median()

print(median_fare_non_survivors)
```